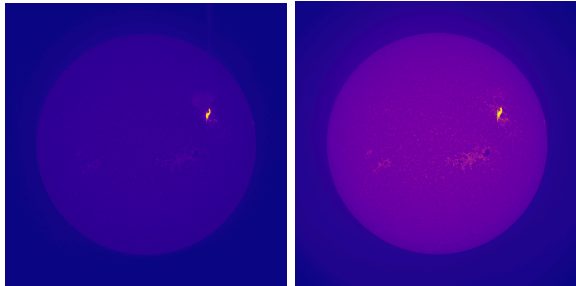


Report 1.1 (the only question):

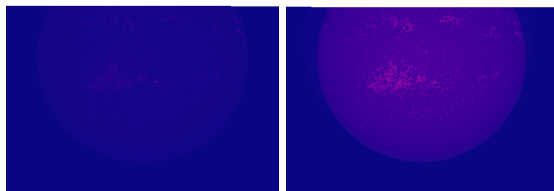
```
● (cs376-env) (base) masa@wireless-10-147-88-181 numpy % python run.py --allwarmups
Running w1
Running w2
Running w3
Running w4
Running w5
Running w6
Running w7
Running w8
Running w9
Running w10
Running w11
Running w12
Running w13
Running w14
Running w15
Running w16
Running w17
Running w18
Running w19
Running w20
Ran warmup tests
20/20 = 100.0
Grade -> 100.0
● (cs376-env) (base) masa@wireless-10-147-88-181 numpy % python run.py --alltests
Running t1
Running t2
Running t3
Running t4
Running t5
Running t6
Running t7
Running t8
Running t9
Running t10
Running t11
Running t12
Running t13
Running t14
Running t15
Running t16
Running t17
Running t18
Running t19
Running t20
Ran all tests
20/20 = 100.0
Grade -> 100.0
❖ (cs376-env) (base) masa@wireless-10-147-88-181 numpy %
```

Report 2.1 (pictures, 2 points):

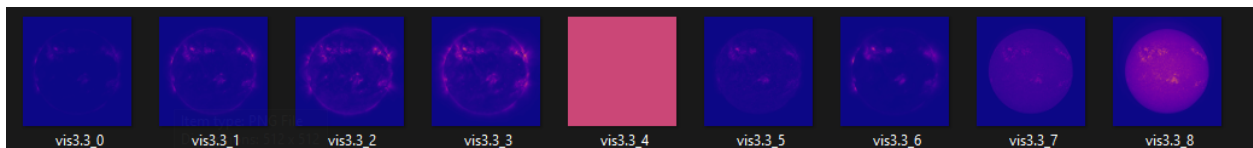
I used the $p^{0.5}$ option, and here are the images of channels 7 and 8 generated with the option.

**Report 2.2 (some pictures, 2 points)**

Here are the images of channels 7 and 8 generated after using the `vmin` and `vmax` options of `plt.imshow`.

**Report 2.3 (3 points):**

I implemented the `colormapArray` function using `np.nanmin`, `np.nanmax` for computing `vmin` and `vmax`. I normalized `X` using the provided formula, `vmin`, and `vmax`. As instructed in the beware sections, I clamped the indices to `[0, N-1]` and scaled the `rgb` values from `[0, 255]`. I mapped them to mid-color index `N//2`, so the visualization does not crash.

**Report 3.1 (3 points):**

My code uses nested for loops that iterate over every pixel, calling `quantize` for each one, so high-resolution images in the gallery require more iterations, causing the code to be extremely slow.

Report 3.2 (3 points):

It seems that the bright areas in the original image appear dark in the false color image, and dark areas appear bright (yellow), and this is because the palette maps index 0 -> 1.0 (white) and index 1 -> 0.0 (black), so low palette indices correspond to high intensity values.

Report 3.3 (2 pictures, 4 points):

The results are created from the `quantizeNaive` algorithm using the command, `python dither.py gallery200 results quantizeNaive`.

(input image -> output image)



(aep.jpg_orig.png)



(rocks.jpg_orig.png)

Report 3.4 (1-2 sentences, 3 points):

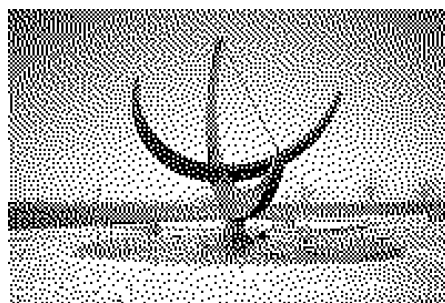
Dithering works because it intentionally adds small, controlled noise which gets spread into some specific patterns our eyes average out, which makes the colors look smoother.

Report 3.5 (3 pictures, 12 points):

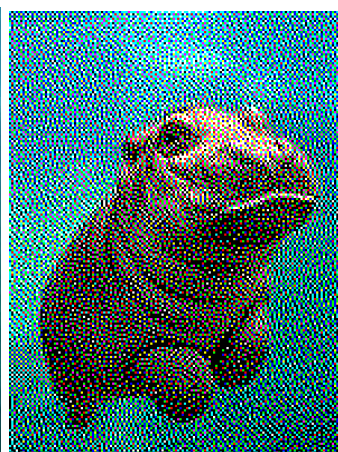
The results are created from the quantizeFloyd algorithm using the command, `python dither.py gallery200 results quantizeFloyd`.

(input images -> output images, including (aep.jpg, babyhippo.jpg, and cosmos.jpg))





Report 3.6 (3 pictures, 5 points):



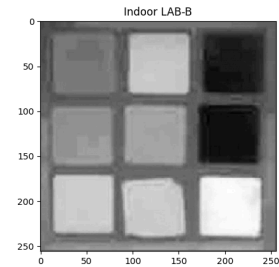
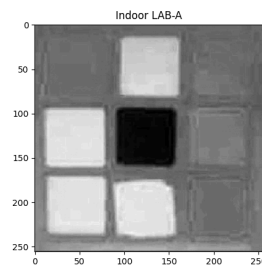
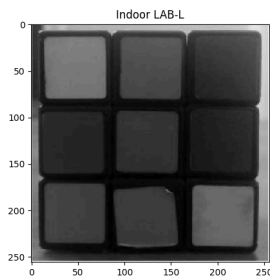


Report 3.7 (optional – 2 extra points):

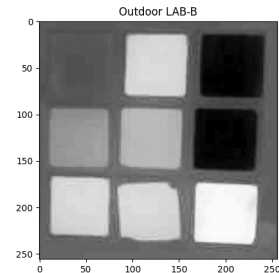
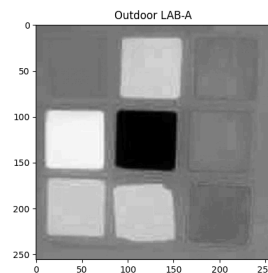
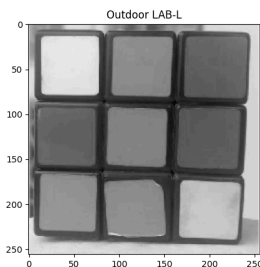
I added mychoice.jpg under the dither directory. I chose the image created with the quantizeFloyd function in a greyscale

Report 4.1 (5 pts):

Indoor LAB images (In the order of: L -> A -> B)



Outdoor LAB images (In the order of: L -> A -> B)



I am convinced that LAB better separates the illuminance than RGB. This is because LAB factorizes color into a lightness channel and two chromatic channels. Thus, when lighting intensity changes, most of the variation shows up as a shift in L with comparatively smaller changes in A and B, which means hue information status more stable. On the other hand, RGB mixes brightness and color together, making it harder to isolate lightening from actual color/hue.